

Nível de Ligação

Redes de Computadores I **LETI/MIETI**

3º ano, 1º semestre
2024/2025

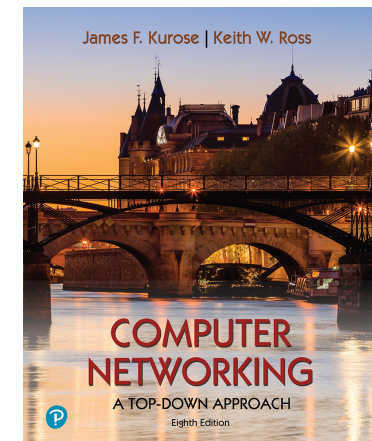
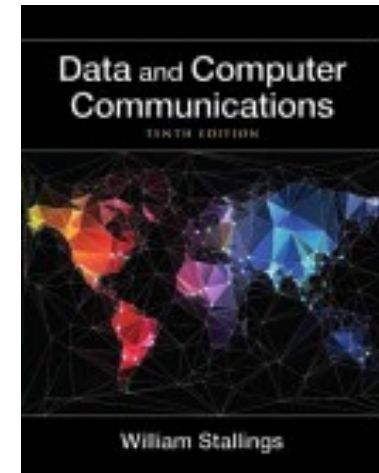


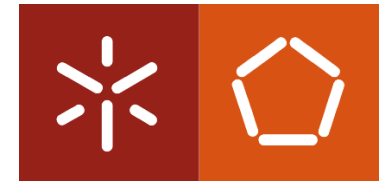
Sumário



- Funções do Nível de Ligação Lógica
- Controlo de Fluxo
 - *Stop and Wait, Sliding Window*
- Detecção de Erros
 - Paridade, *Checksum*, CRC
- Controlo de Erros
 - *Stop and Wait ARQ, Go-Back-N ARQ, Selective-Repeat ARQ*
- Gestão da Ligação
- Técnicas de Acesso ao Meio

**Este módulo é maioritariamente baseado nos livros
Data and Computer Communications, Willian Stallings e
*Computer Networking: A Top Down Approach, Jim Kurose and Keith Ross***

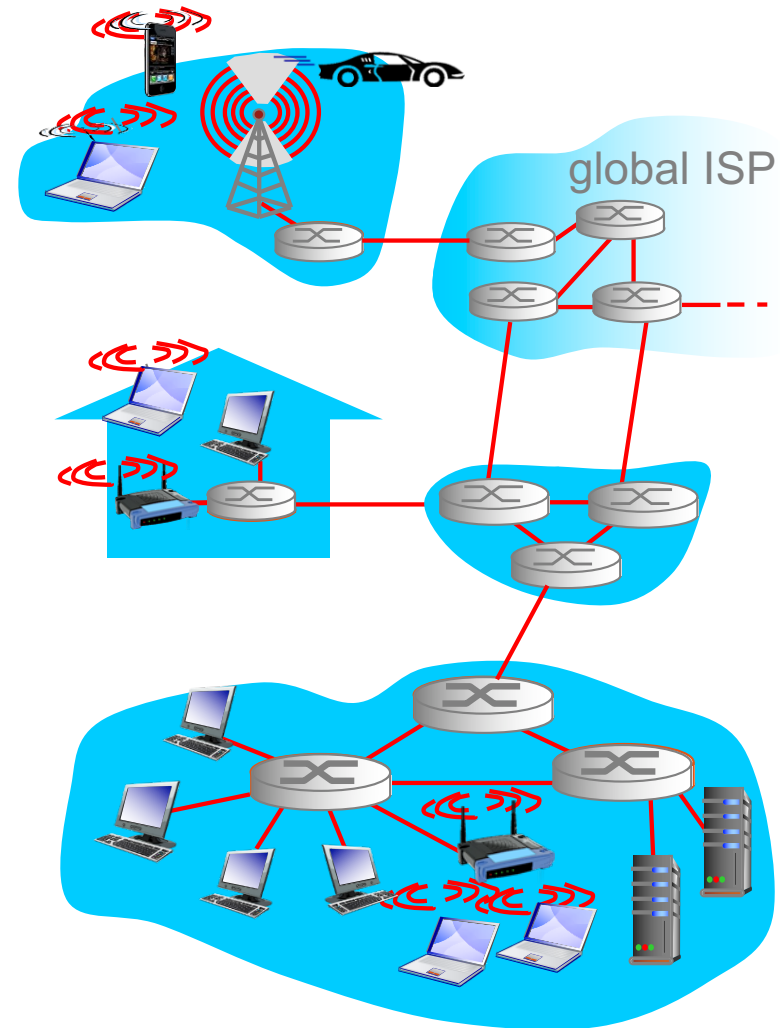




Controlo da ligação de dados

Terminologia:

- Sistemas terminais e encaminhadores: **nós**
- Canais de comunicação que interligam nós adjacentes:
ligações
 - Ligações cabladas ponto-a-ponto
 - Ligações sem fios
 - Redes Locais por difusão
- Pacote de nível 2: **trama** encapsula datagrama



Controlo da ligação de dados



- A existência de ligações físicas e a transmissão de sinais analógicos ou digitais, por si só, não garantem a comunicação de dados entre entidades residentes em diferentes sistemas.
- A troca de dados entre entidades que pretendem comunicar deve ser regulada a fim de se criar um contexto comum e um sincronismo entre elas.
- As *regras* resultantes constituem o que se designa por *protocolo de comunicação*.
- Os *protocolos de ligação lógica* ou *ligação de dados* constituem o primeiro nível de troca ordenada, controlada e fiável de dados entre sistemas interligados por meio de uma ligação física.

Controlo da ligação de dados

introdução: funções distintivas dos níveis físico e lógico



Nível físico

- envio de um sinal sobre um meio de transmissão
- sincronismo (nível do bit)
- codificação de linha
- modulação do sinal
- multiplexagem física
- interface com o meio

Nível de ligação lógica

- estrutura das tramas
- configuração e acesso à linha
- endereçamento
- controlo de fluxo
- controlo de erros
- gestão da ligação (controlo da troca de dados)

O **nível de ligação lógica** tem a responsabilidade de transferir um datagrama de um nó para outro nó adjacente.

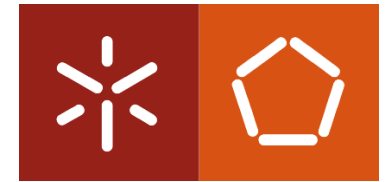
Controlo da ligação de dados

principais funções de um protocolo de ligação

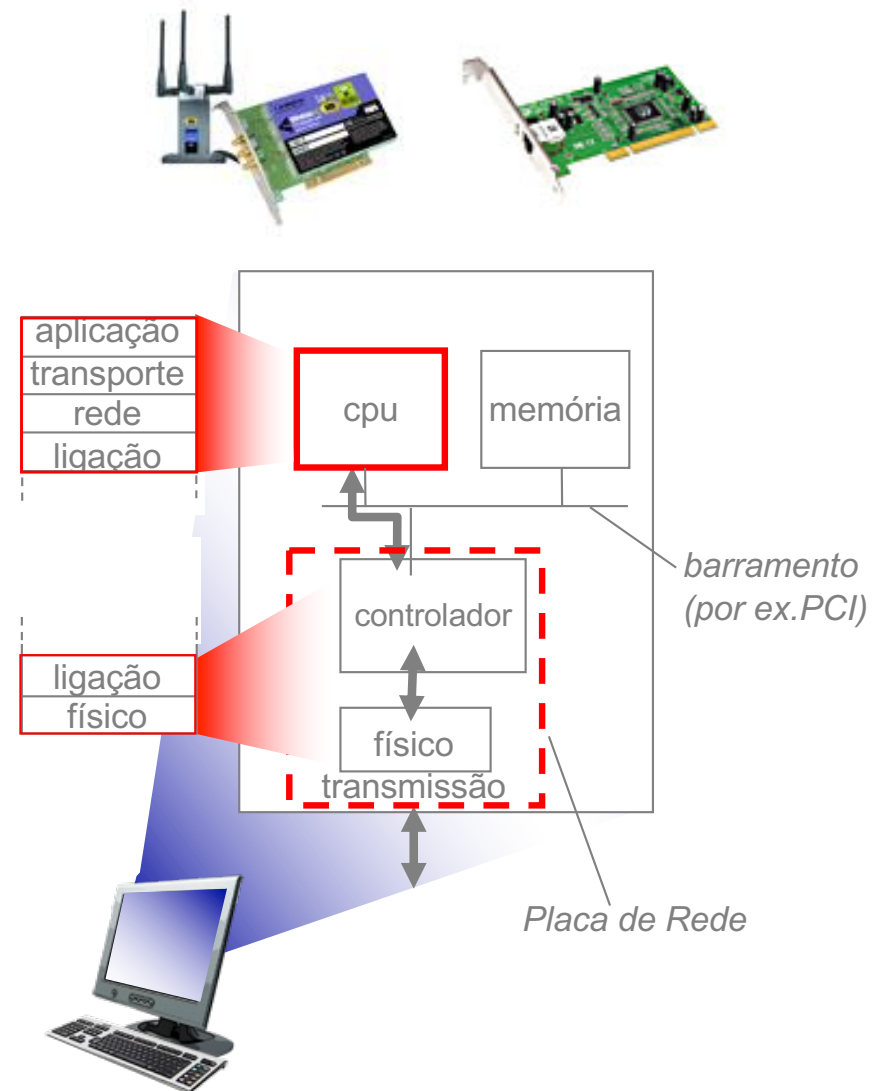


- **definição da trama** - formato da unidade de dados (PDU)
- **configuração da linha** - considera a topologia, define a disciplina de acesso à linha e a sua duplexidade
- **endereçamento** - identifica os interfaces das estações que podem enviar e receber tramas
- **controlo de fluxo** - regula a cadência de tramas enviadas
- **controlo de erros** - detecta erros de transmissão e executa procedimentos de recuperação
- **gestão da ligação** - define como se faz o estabelecimento, a manutenção e a terminação da associação lógica.

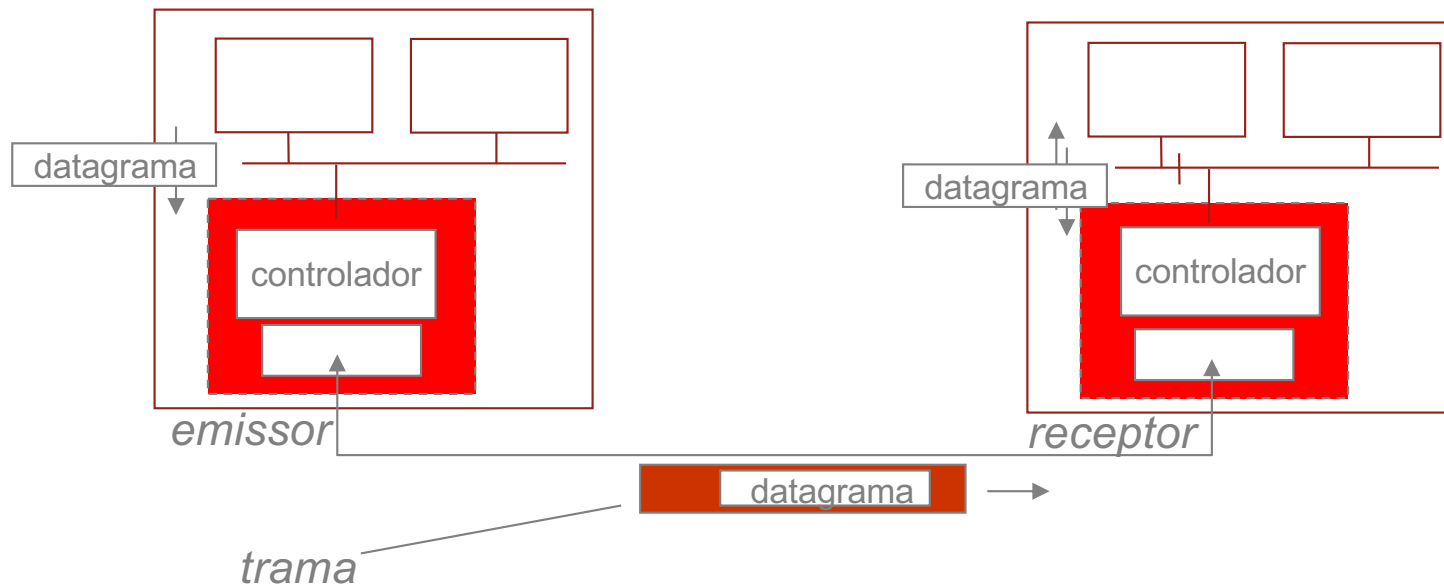
Implementação da camada de ligação



- Em todos os nós (sistemas terminais, routers, *switches*)
- O nível de ligação é implementado na interface de rede (placa Ethernet, placa Wi-Fi, etc.)
 - Implementa habitualmente a camada física e a de ligação lógica
 - Está conectada ao barramento do dispositivo
 - É uma combinação de hardware, software e firmware



Implementação da camada de ligação



❖ emissor:

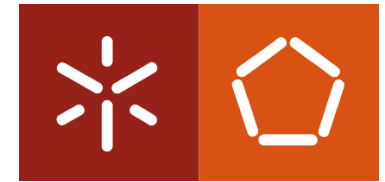
- encapsula datagrama na trama;
- adiciona cabeçalho que permite o controlo de fluxo e a deteção e controlo de erros;

❖ receptor:

- efetua controlo de fluxo, deteta e controla os erros;
- extrai datagrama e entrega-o à camada de rede

Controlo da ligação de dados

controlo de fluxo

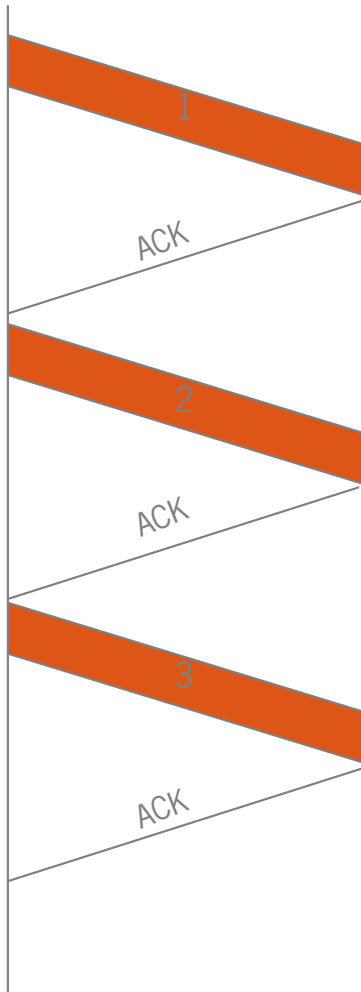


- Técnica para assegurar que a estação que transmite não sobrecarrega a que recebe.
- Em geral, a existência de buffers na estação de receção reduz, mas não elimina, a necessidade de controlar o fluxo.
- Técnicas mais comuns de controlo de fluxo:
 - *stop-and-wait* (para-e-espera)
 - *sliding window* (janela deslizante)

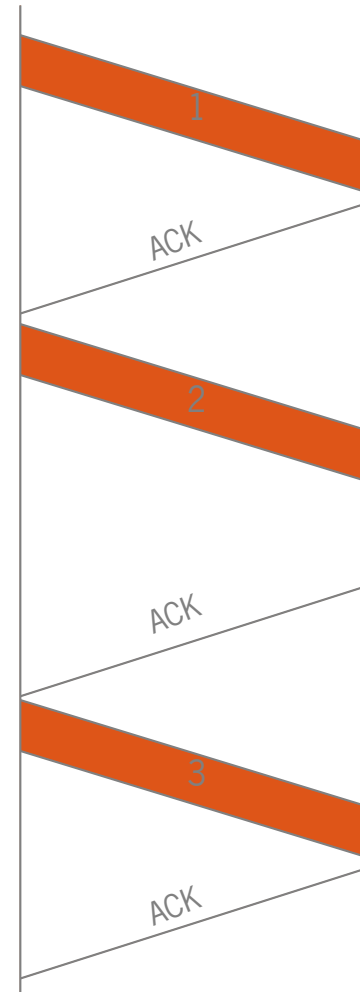
Stop-and-Wait



O emissor envia uma trama logo que recebe a confirmação da anterior.

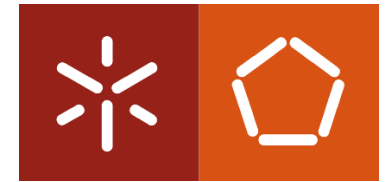


O receptor está disponível para receber mais tramas, pelo que envia a **confirmação de imediato**.



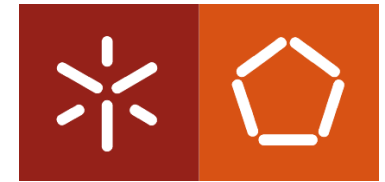
O receptor está ocupado, pelo que **adia o envio da confirmação**.

Stop-and-Wait



- Após a transmissão de uma trama, a fonte aguarda confirmação da sua recepção (ACK) antes de transmitir a trama seguinte.
- A recepção pode parar o fluxo de dados suspendendo temporariamente as confirmações.
- Esta técnica funciona bem quando uma mensagem é fragmentada em poucas tramas de grande dimensão. Contudo, se o tamanho das tramas é grande...
 - é maior a probabilidade de erro na trama,
 - é maior a ocupação de recursos (buffers, processadores),
 - o desempenho da ligação tende a piorar, principalmente nas ligações multiponto
- O *stop-and-wait* torna-se inadequado se a mensagem for fragmentada em pequenas tramas.

Stop-and-Wait

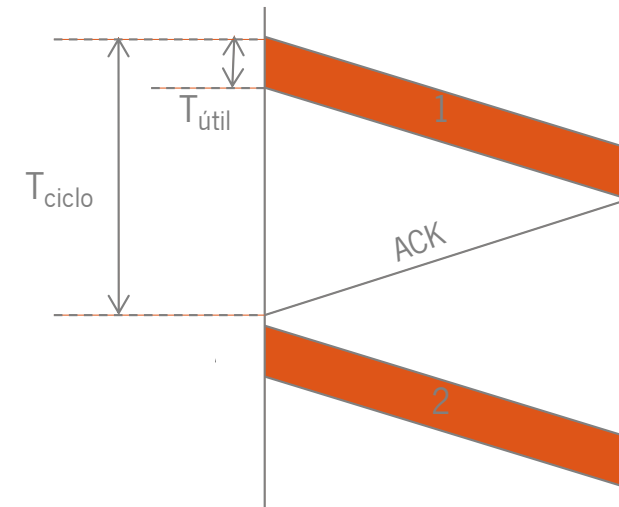


- Em determinadas circunstâncias, o mecanismo *Stop-and-Wait* pode penalizar muito a **utilização da ligação**.
- A utilização mede a percentagem de tempo que a ligação está efectivamente a ser utilizada para transmitir dados úteis.
- Exemplo:

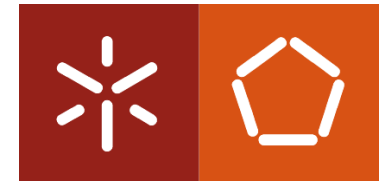
- $T_{\text{transmissão}} = 1\text{ms}$
- $T_{\text{propagação}} = 10\text{ms}$
- $T_{\text{transmissão_conf}} \approx 0\text{ ms}$

copiar formulas

- $U = T_{\text{útil}} / T_{\text{ciclo}}$
- $U = T_{\text{transmissão}} / (T_{\text{transmissão}} + 2 \times T_{\text{propagação}})$
- $U = 1 / (1 + 2 \times 10) \approx 4,76\% !!!$



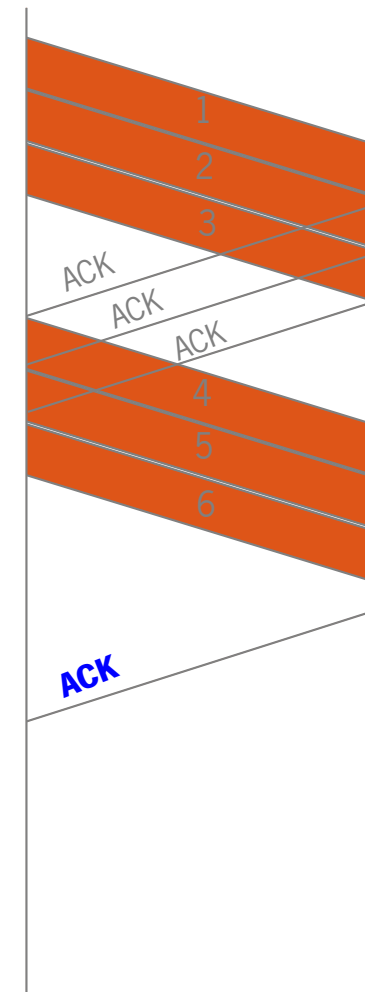
Sliding Window



- Os mecanismos baseados em **janela deslizante** (*sliding window*) permitem fazer uma utilização mais eficiente da ligação.
- Dada uma abertura da janela W , o emissor pode enviar até W tramas sem ter que esperar pela confirmação.
- Implica sequenciação das tramas (n bits).

O emissor pode enviar até W tramas sem esperar pela confirmação.

$W = 3$



O receptor pode enviar as confirmações de imediato, ou **adiá-las** para reduzir o fluxo.

Sliding Window



- Porquê “janela” deslizante?

Buffer do emissor: tramas à espera de serem enviadas:

	1	2	3	4	5	6	7	8	9			
--	---	---	---	---	---	---	---	---	---	--	--	--

Se $W = 3$, então só podem ser enviadas 3 tramas:

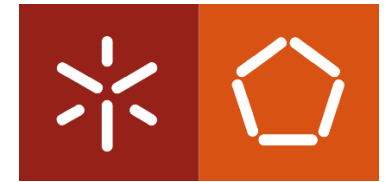
	1	2	3	4	5	6	7	8	9			
--	---	---	---	---	---	---	---	---	---	--	--	--

Assim que chega a primeira confirmação, a janela desliza de uma unidade, permitindo o envio de mais uma trama (a número 4):

	1	2	3	4	5	6	7	8	9			
--	---	---	---	---	---	---	---	---	---	--	--	--

→

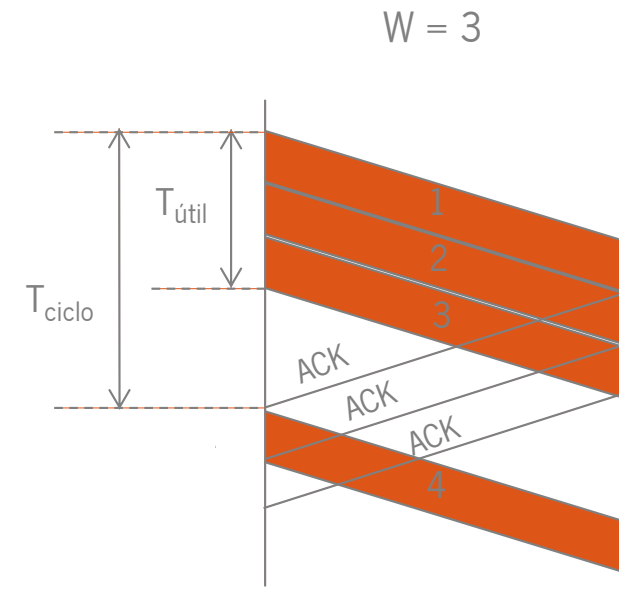
Sliding Window



- O mecanismo de janela deslizante permite uma maior utilização da ligação:
- Exemplo:

- $T_{\text{transmissão}} = 1\text{ms}$
- $T_{\text{propagação}} = 10\text{ms}$
- $T_{\text{transmissão_conf}} \approx 0\text{ ms}$

[copiar formulas](#)



- $U = T_{\text{útil}} / T_{\text{ciclo}}$
- $U = 3 \times T_{\text{transmissão}} / (T_{\text{transmissão}} + 2 \times T_{\text{propagação}})$
- $U = 3 \times 1 / (1 + 2 \times 10) \approx 14,3\%$

(3 vezes maior do que com o *stop-and-wait*)

Controlo da ligação de dados

controlo de fluxo - utilização da ligação



- A utilização (rendimento da ligação) depende de W e do parâmetro a
- O parâmetro a é a razão entre o tempo de propagação e o tempo de transmissão

$$a = t_{\text{prop}} / t_{\text{trama}}$$

[copiar para os resumos](#)

$$a = (d/v) / (L/r)$$

$$a = rd / vL$$

d - distância (m); v - velocidade de propagação (m/s);

L - comprimento trama (bits); r - ritmo de transmissão (bps)



- **Ligações satélite**

Valores típicos:

$$t_{\text{prop}} = 270 \text{ ms}$$

$$L = 4000 \text{ bits}; \quad r = 56 \text{ Kbps} \quad \Rightarrow \quad t_{\text{trama}} = 71 \text{ ms}$$

$$a = 3.8$$

Para $125\mu\text{s} < t_{\text{trama}} < 6\text{ms}$ os valores de a são $a \gg 1$

- **Redes locais**

Valores típicos:

$$0.1 < d < 10 \text{ Km}; \quad v = 2 \times 10^8 \text{ m/s}$$

$$L = 500 \text{ bits}; \quad 0.1 < r < 10 \text{ Mbps}$$

Neste caso o parâmetro a tem um valor pequeno, $a \ll 1$

Controlo da ligação de dados

controlo de fluxo - utilização da ligação



- **Stop-and-Wait**

- Supondo que uma mensagem é enviada numa sequência de *frames* $f_1, f_2, \dots, f_n$, então o tempo total para enviar a mensagem pode ser expresso como $t_{\text{total}} = n * t_{\text{frame}}$, onde t_{frame} é o tempo necessário para enviar uma frame e receber um ack.

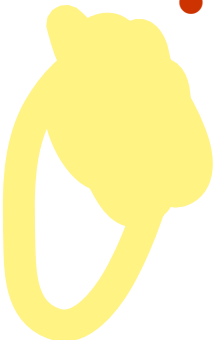
$$t_{\text{frame}} = t_{\text{transFrame}} + t_{\text{prop}} + t_{\text{proc}} + t_{\text{transAck}} + t_{\text{prop}} + t_{\text{proc}}$$

Assumindo que $t_{\text{proc}} \approx 0$ e $t_{\text{transAck}} \approx 0$

$$t_{\text{total}} = n * (t_{\text{transFrame}} + 2 * t_{\text{prop}})$$

- A **Utilização** da ligação é a fração do tempo total que é útil, i.e., que é utilizado a transferir tramas de dados, $U = t_{\text{util}} / t_{\text{total}}$:

$$U = n * t_{\text{transFrame}} / n * (t_{\text{transFrame}} + 2 * t_{\text{prop}}) = 1 / (1 + 2a)$$





- ***Sliding Window (Janela Deslizante)***

Exemplo: ligação *full-duplex* entre duas estações A e B

- **Caso 1** - A estação A transmite continuamente. A confirmação de chegada da trama 1 ocorre **antes** da janela se fechar, então

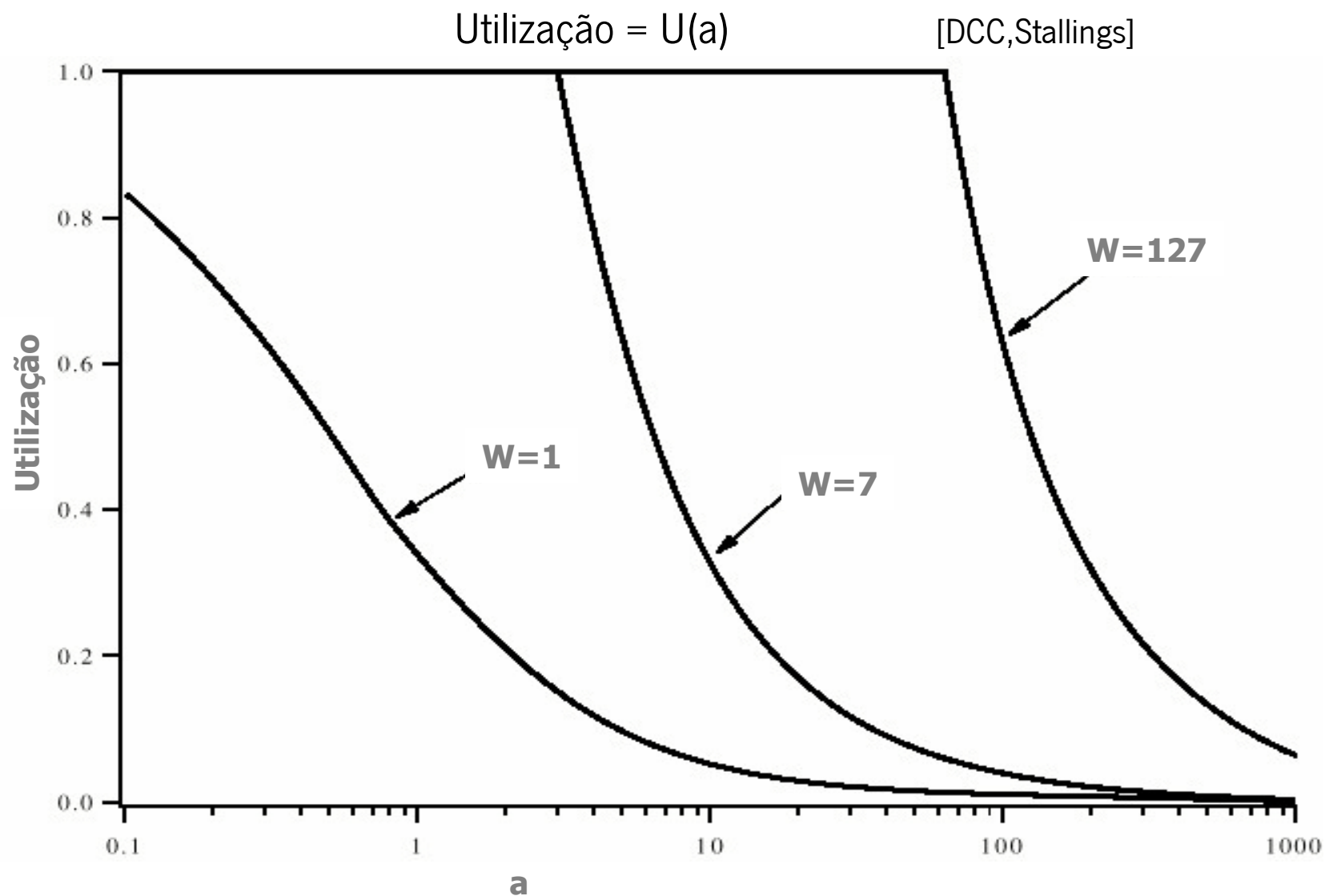
$$U = 1 \quad \text{se} \quad W \geq 2a + 1$$

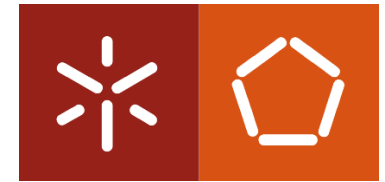
- **Caso 2** - A estação A tem a janela fechada em $t_0 + W$ e não pode enviar tramas até $t_0 + 2a + 1$ (chegada do primeiro ACK), então

$$U = W / (2a + 1) \quad \text{se} \quad W < 2a + 1$$

Controlo de ligação de dados

controlo de fluxo - utilização da ligação



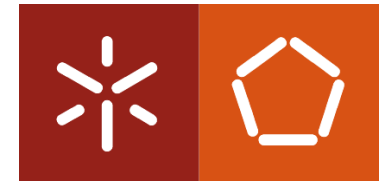


Para pensar:

- **Considere uma ligação *full duplex* ponto-a-ponto entre dois computadores, A e B, com as seguintes características:**
 - Taxa de transmissão: 100 Mbps
 - Tamanho fixo da trama de 500 bytes
 - Distância entre o emissor e o recetor: 28 Km
 - Tamanho dos ACKs desprezável
 - Velocidade de propagação do sinal: $2,8 \times 10^8$ m/s

Suponha que para fazer o controlo de fluxo se pretende usar Sliding Window. Qual seria o tamanho mínimo da janela a ser usado nesta ligação, para garantir uma utilização máxima do meio de transmissão?

Controlo de erros (ex. 1a)



A seguinte mensagem foi recebida por um construtor:

De:

Banhos maravilha, Lda.

Rua da água, 87

8888-888 Rio de baixo

Para:

Constroi já, Lda.

Avenida do cimento, 156

9999-999 Andaime

Serve esta carla para apresentar o nosso melhor orçamento para o fornecimento e instalação de uma sauna, tal como coxbinado.

Orçamento: 6 396,45 € + iva à taxa legal em vigor

Comente o conteúdo desta mensagem.

Controlo de erros (ex. 1b)



A mensagem realmente enviada foi:

De:

Banhos maravilha, Lda.

Rua da água, 37

8888-888 Rio de baixo

Para:

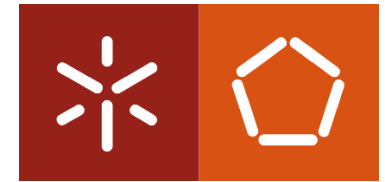
Constroi já, Lda.

Avenida do ceimento, 156

9999-999 Andaime

Serve esta carta para apresentar o nosso melhor orçamento para o fornecimento e instalação de uma sauna, tal como combinado.

Orçamento: 2 396,45 € + iva à taxa legal em vigor



Controlo de erros

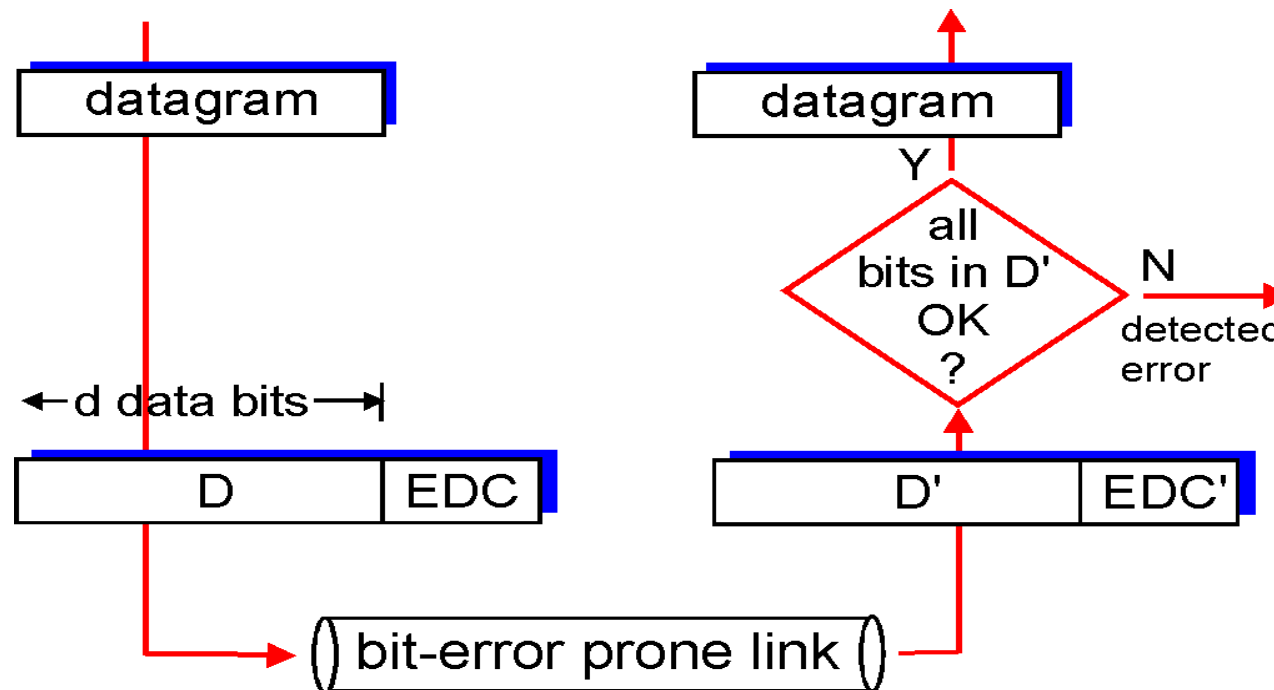
- **Acções perante situações de erro:**
 - Detectar a ocorrência de erros
 - Rejeitar os dados com erros
 - Ignorar que ocorreram erros
 - Tentar corrigir os erros
 - A partir dos dados com erros: FEC – *Forward Error Correction*
 - Por retransmissão automática
 - Não detectar a ocorrência de erros
 - Utilizar os dados tal como são recebidos

Controlo de ligação de dados

detecção de erros



- A cada trama (D: Datagram), o transmissor adiciona um número de bits (**EDC: Error Detection and Correction bits**) que será usado pelo receptor para detecção de erros.

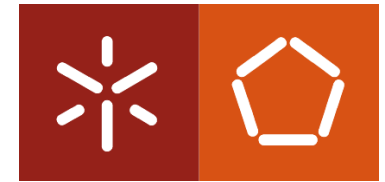


Controlo de ligação de dados

detecção de erros

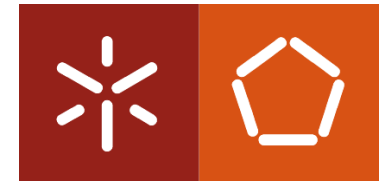


- A detecção de erros **não é 100% fiável**: o protocolo pode não detectar erros!!
- **Probabilidade de erro residual** - probabilidade de existirem erros em número superior aos que é possível detectar pelo mecanismo utilizado para o efeito.
- **Quanto maior for o número de bits usados no campo EDC, maior é a probabilidade de sucesso na detecção e correcção de erros**



Detecção de erros

- **Algumas técnicas de detecção de erros:**
 - Códigos de blocos
 - Paridade simples
 - Paridade vertical/longitudinal
 - Somas de controlo (*checksums*)
 - Códigos cíclicos
 - CRCs – *Cyclic Redundancy Check*



Paridade simples

- **A cada bloco de 7 ou 8 bits acrescenta-se um bit para deteção de erros:**
 - Paridade par: o número total de 1's deve ser par:
00101110 0
 - Paridade ímpar: o número total de 1's deve ser ímpar:
00101110 1



Paridade simples

- **Deteção de erros (paridade par):**

- Sequência transmitida:

00101110 0 (o nº de 1's é par)

- Sequência recebida:

00101010 0 (o nº de 1's é ímpar!)

- Sequência recebida (2):

10101010 0 (o nº de 1's é par!)

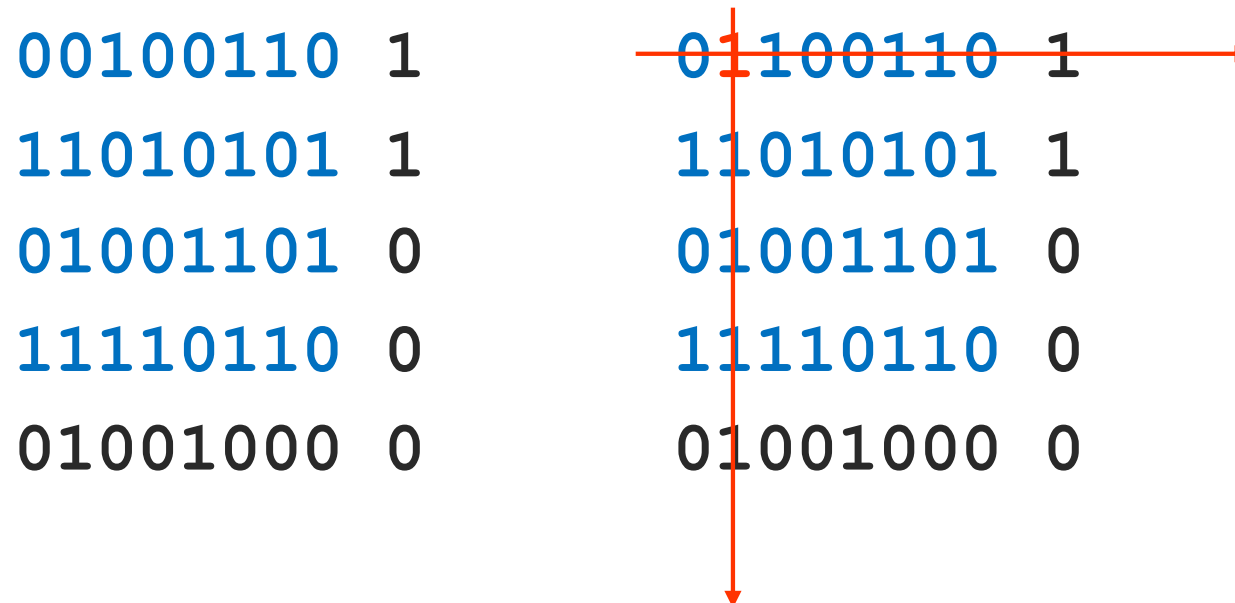
(os erros não foram detetados!)

Erros em número par (2, 4, 6, 8) não são detetados.



Paridade vertical/longitudinal

- Os dados são agrupados em blocos de $N \times M$ bits:



Ainda assim há certos padrões de erros que não são detetados.

Somas de controlo (*checksums*)



- Os dados são agrupados em blocos de N bits por linha e por um número variável de linhas:

0010011011010101 ... 0100110111110110

00100110

11010101

.....

01001101

11110110

01001000 <- soma de controlo

0010011011010101 ... 010011011111011001001000



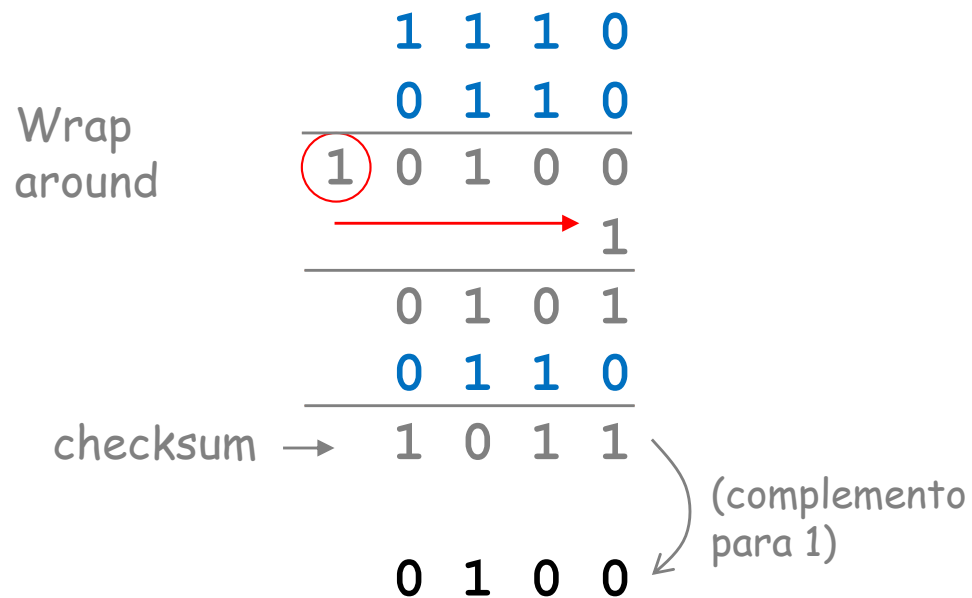
Controlo de ligação de dados

detecção de erros: soma de verificação (checksum)



Exemplo: $k=4$, mensagem = 1110 0110 0110

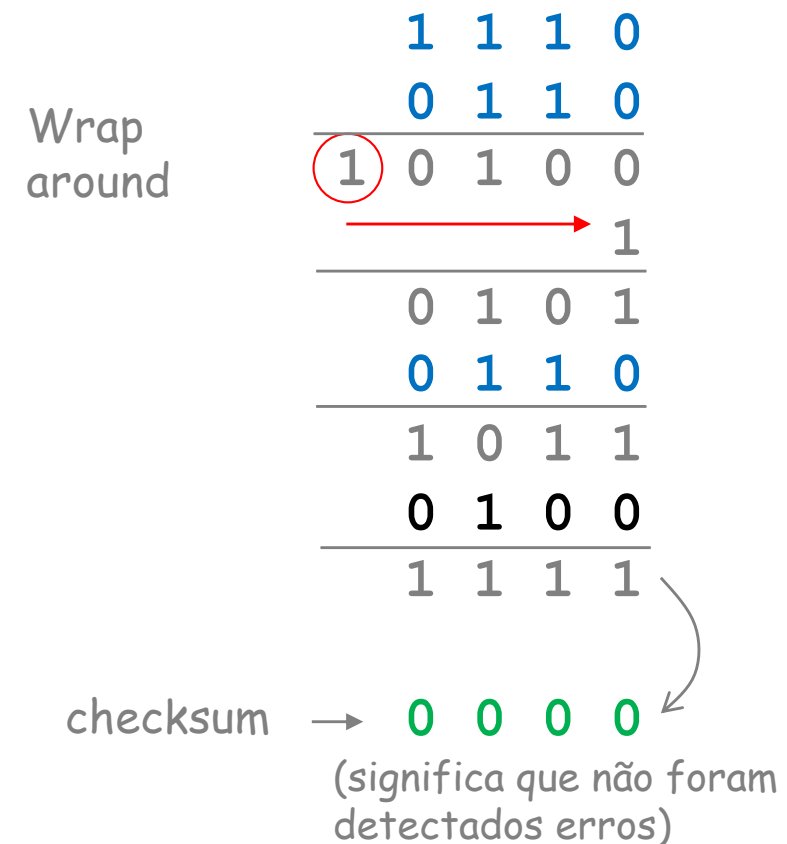
No emissor:

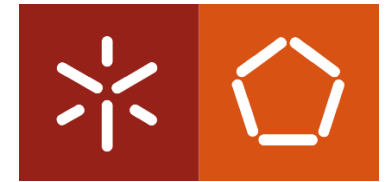


Mensagem a enviar:

1110011001100100

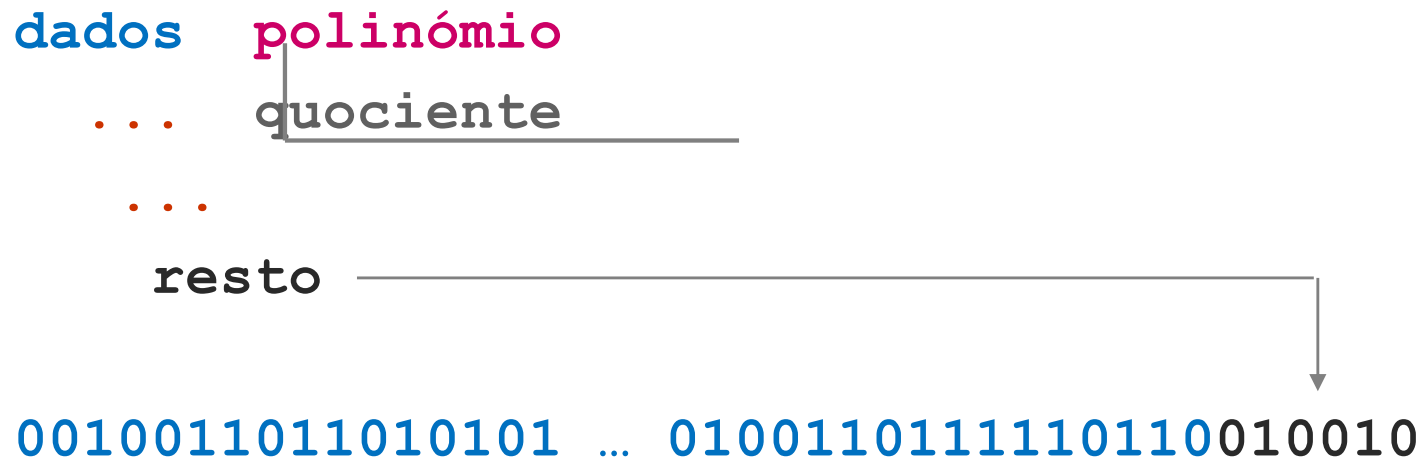
No receptor:





- **Cyclic Redundancy Check:**

- A sequência a proteger é dividida por um número pré-determinado (polinómio gerador)
- O resto da divisão é enviado como redundância



Controlo de ligação de dados

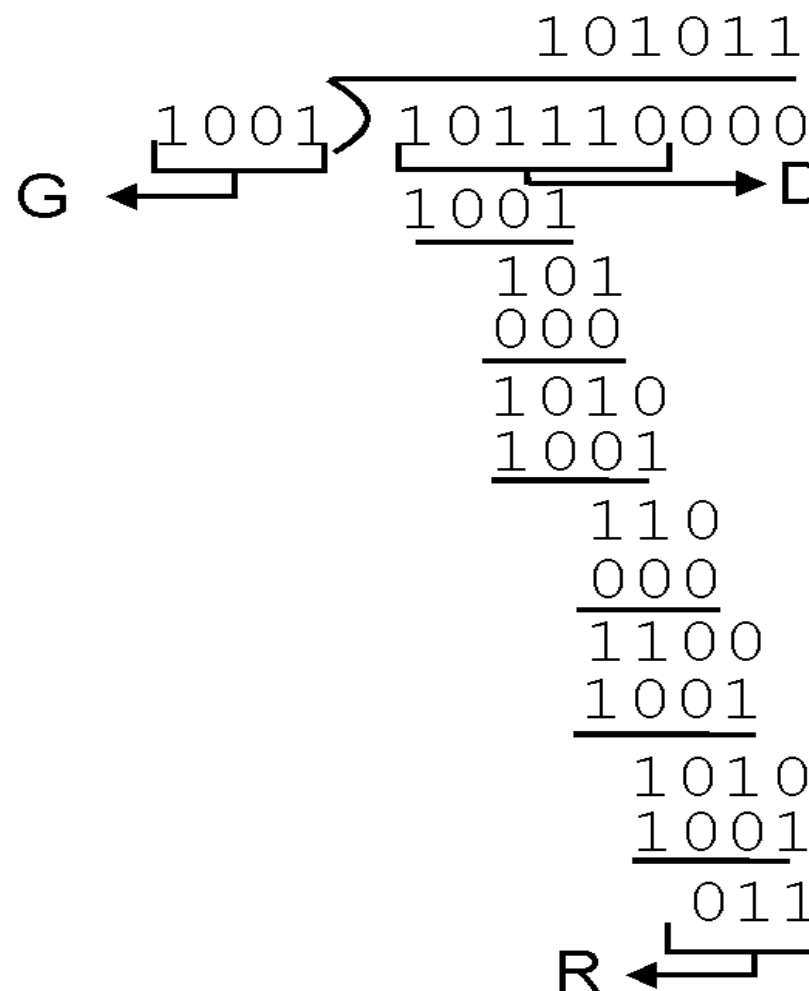
detecção de erros: verificação de redundância cíclica (CRC)



Exemplo no emissor:

Mensagem a enviar: 101110

Mensagem enviada: 101110011



Controlo de ligação de dados

detecção de erros: verificação de redundância cíclica (CRC)



- O processo CRC é, em geral, expresso através de polinómios de uma variável, com coeficientes binários.

Exemplo de um polinómio gerador $G(x)$:

CRC-32: $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$

100000100110000010001110110110111 (33bits, para dar resto 32)

normalizado para transmissão síncrona ponto-a-ponto (IEEE 802.x)

- Pode falhar se o número de erros for superior à capacidade de deteção do código
- Consegue detetar todos os **n-k** (sendo n-k, o grau do polinómio gerador), erros consecutivos, e mesmo rajadas com mais de **n-k** erros embora com probabilidade **1 - 0.5^(n-k)**



- **Técnicas de controlo de erros:**
 - FEC (*Forward Error Correction*):
 - Códigos de blocos
 - Códigos cíclicos
 - Retransmissão automática (*ARQ – Automatic Repeat reQuest*)
 - *Stop-and-wait*
 - Janela deslizante
 - *Go-back-N*
 - *Selective Repeat*

Correção de erros

Stop-and-Wait



- ***Stop-and-Wait***

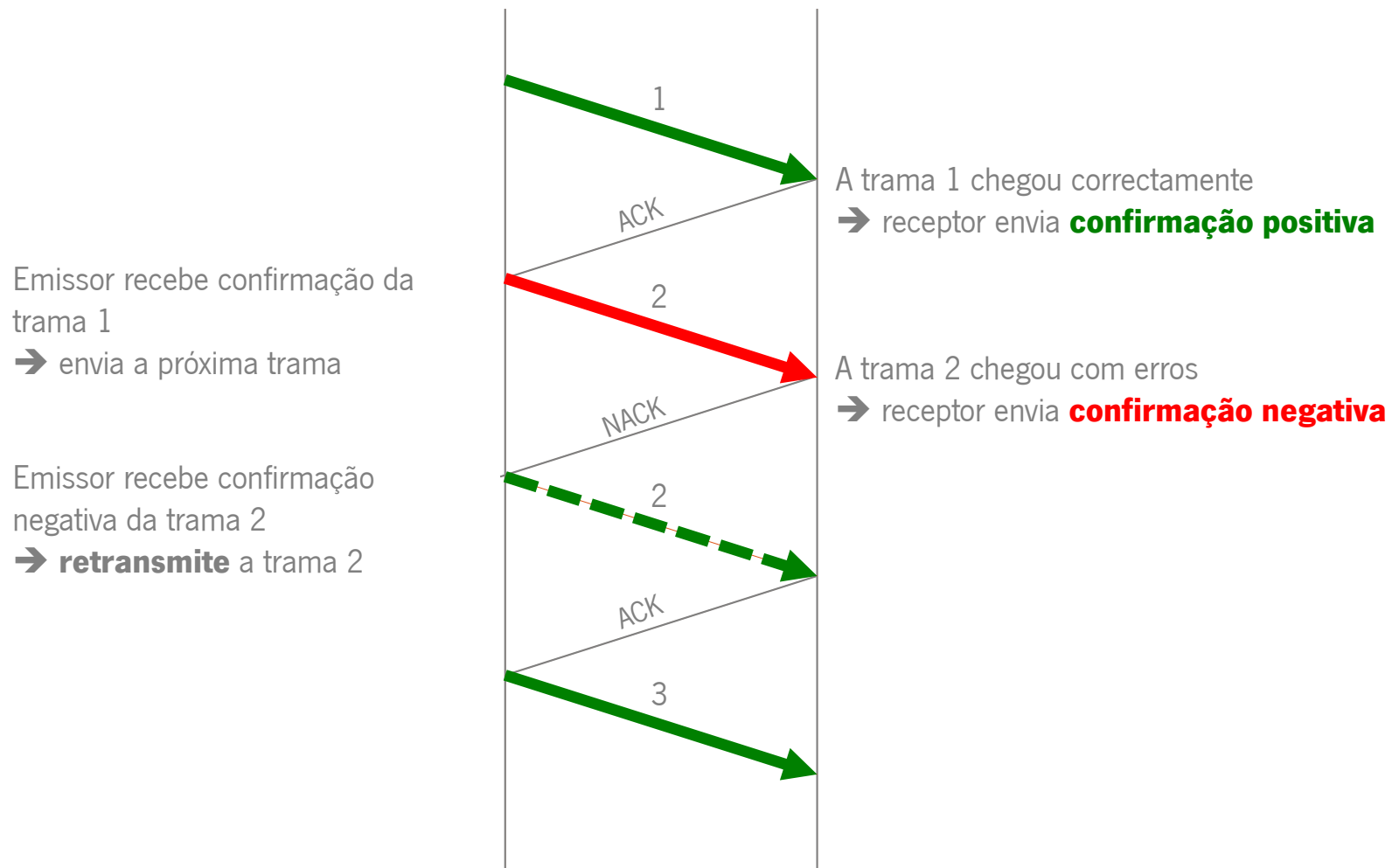
- semelhante à técnica de controlo de fluxo *stop-and-wait*
- transmissor:
 - ativa temporizador e mantém cópia da trama até obter ACK
 - no máximo espera *timeout* até transmitir de novo
- recetor:
 - envia ACK, NAK (pedido explícito) ou *no reply* (pedido implícito)
- **sequenciação necessária para resolver a situação de erro na trama de confirmação (duplicação da trama): um bit chega!**
- vantagem: simples; desvantagem: reduzida eficiência

Correção de erros

Stop-and-Wait



Solução 1: confirmações positivas (ACK) e negativas (NACK)



Correção de erros

Stop-and-Wait



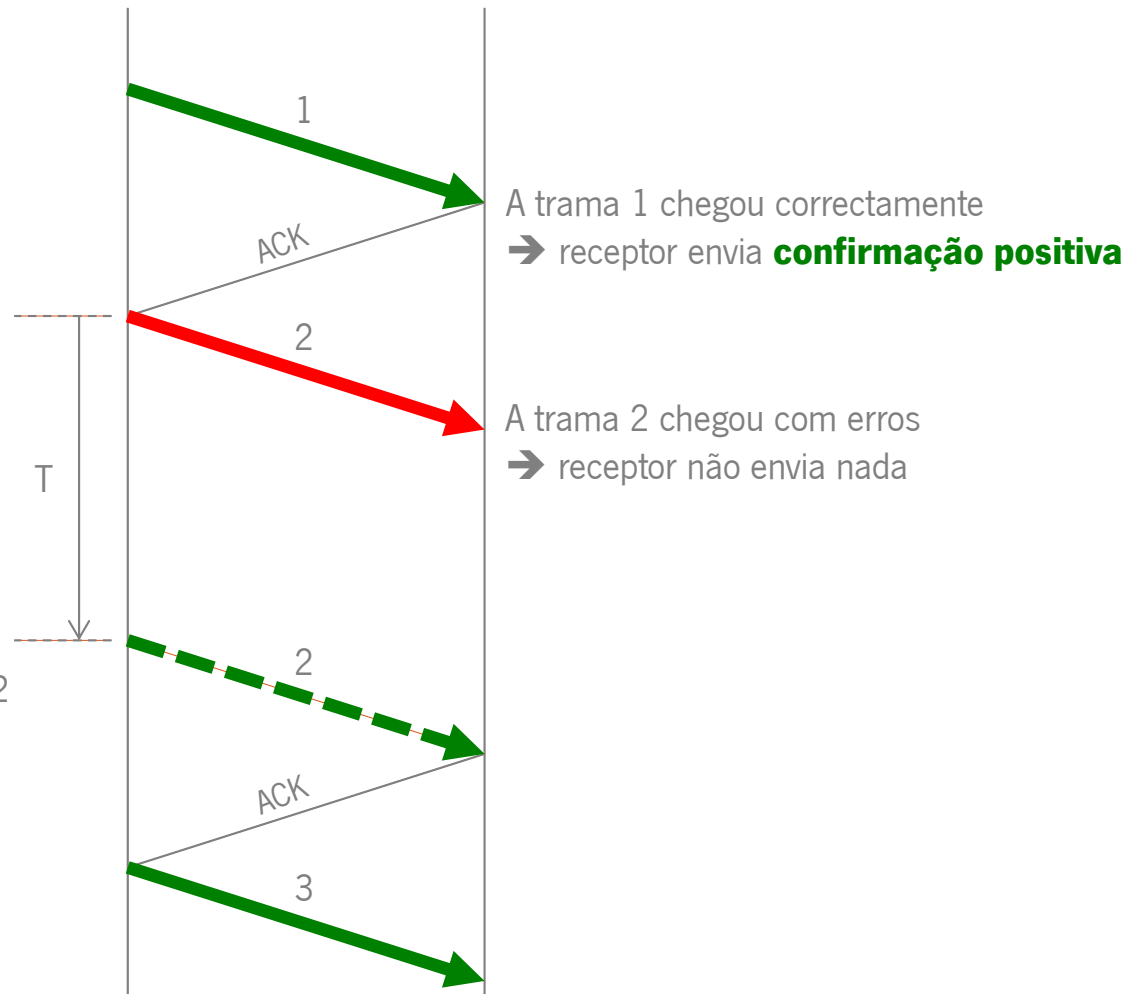
Solução 2: confirmações positivas e temporizadores (I)

Emissor recebe confirmação da trama 1

→ envia a próxima trama

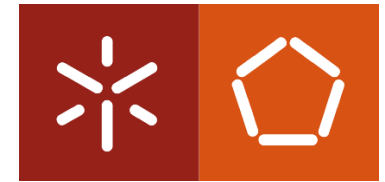
Temporizador da trama 2 expira

→ emissor **retransmite** a trama 2



Correção de erros

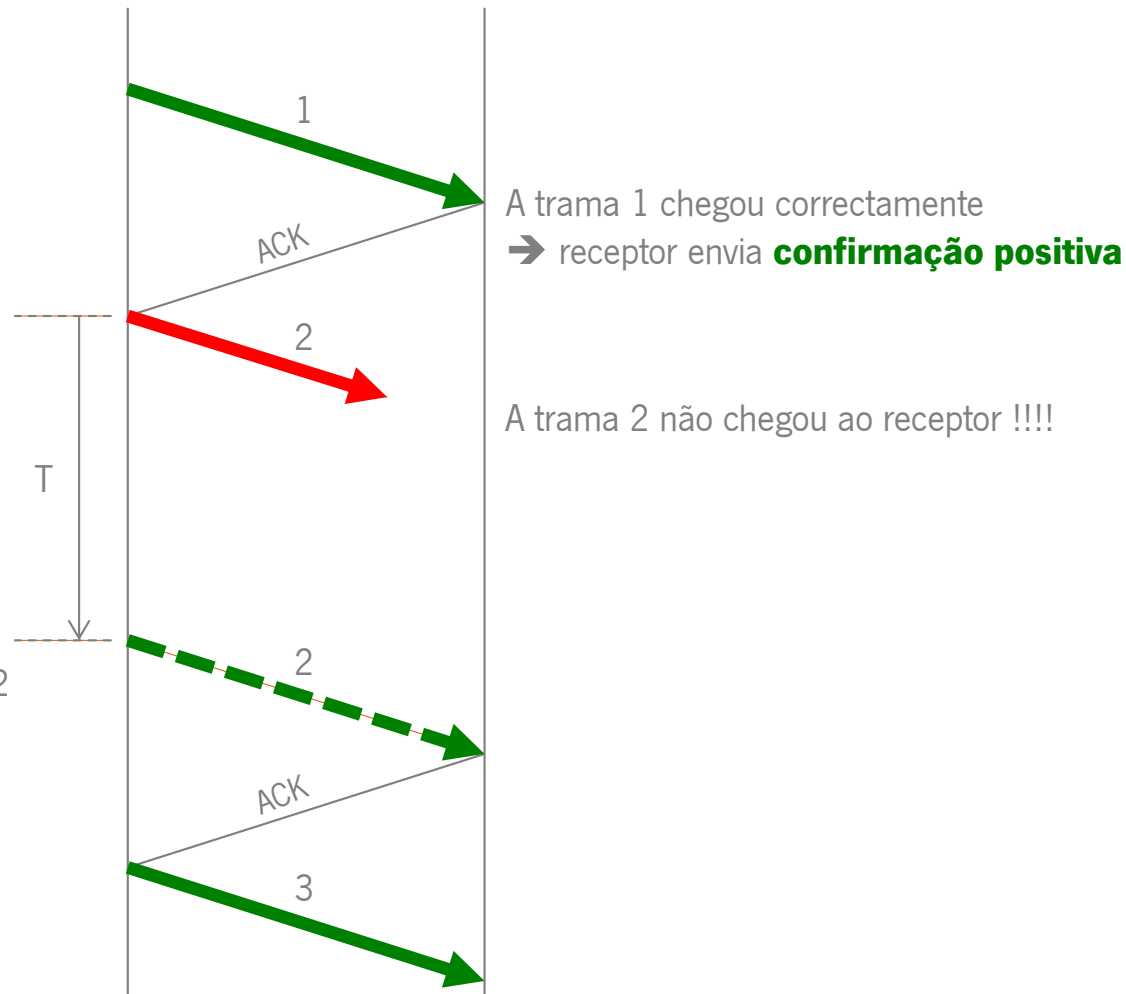
Stop-and-Wait



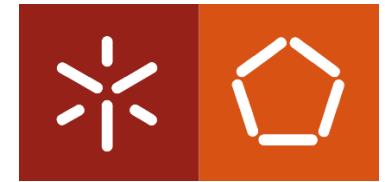
Solução 2: confirmações positivas e temporizadores (II)

Emissor recebe confirmação da trama 1
→ envia a próxima trama

Temporizador da trama 2 expira
→ emissor **retransmite** a trama 2



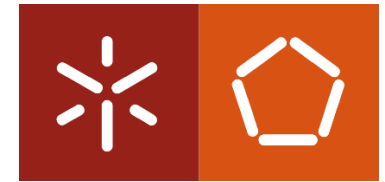
Eficiência do ARQ



- Tal como vimos no caso do controlo de fluxo, o mecanismo *Stop-and-Wait* é pouco eficiente em algumas ligações
- Nestes casos pode fazer-se controlo de erros usando mecanismos baseados em janela deslizante:
 - *Go-Back-N* (mais simples, menos eficiente)
 - *Selective Repeat* (mais complicado, mais eficiente)

Correção de erros

Go-Back-N



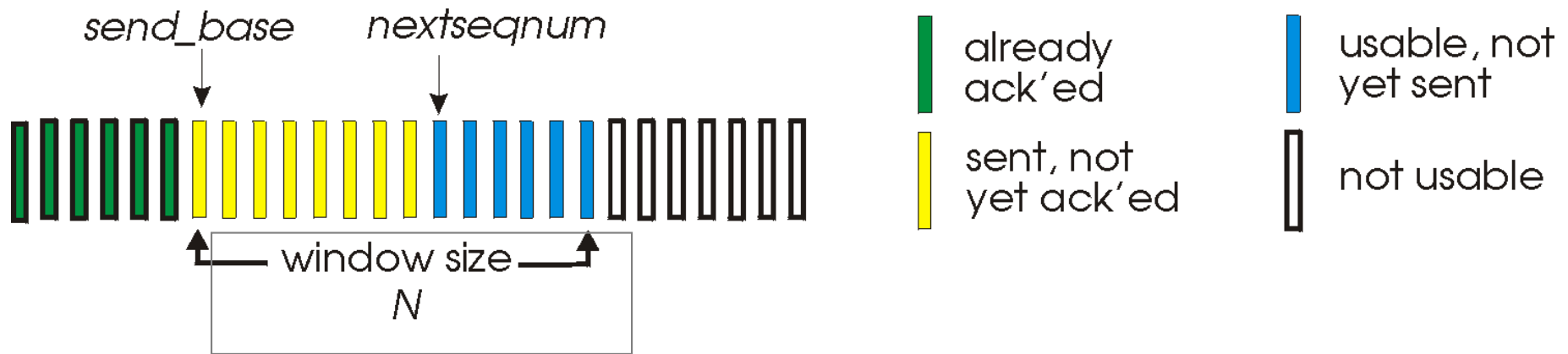
- ***Go-Back-N***

- a falta de sequenciação ou erro na receção implica a retransmissão a partir de uma determinada ordem.
- $W_{\max} = 2^n - 1$, sendo n o número de bits necessários para sequenciação

Exemplos:

- trama t_i corrompida ou perdida
 - B recebeu $t_{(i-1)}$ e deteta erro em t_i ; B envia NAK i
 - t_i é perdida, B recebe $t_{(i+1)}$ e envia NAK i
 - t_i é perdida; A retransmite t_i por *timeout*
- confirmações corrompidas
 - confirmação por ACK seguintes (os ACKs são cumulativos)
 - se A expira, A retransmite t_i e todas as tramas subsequentes

Go-Back-N: emissor



- ACK(i): Confirma todos os pacotes até ao pacote i (os ACKs são cumulativos).
- Tem que usar no mínimo um timer para o pacote mais antigo que foi enviado e não foi ainda confirmado porque os ACKs (e/ou os NAKs) podem ser corrompidos e nesse caso é necessário recuperar por timeout.
- *timeout(n)*: retransmits o pacote n e todos os outros com número de sequência maior que n contidos na janela
- Podem ser usados ou não NAKs

Correção de erros

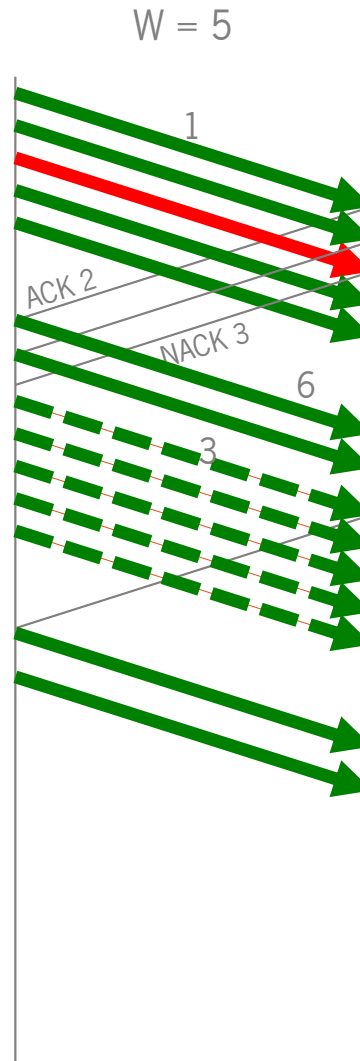
Go-Back-N



Com NACKs

Por cada confirmação positiva que chega
→ emissor envia mais uma trama, até esgotar a janela

Emissor recebe confirmação negativa da trama 3
→ emissor **retransmite** a trama 3, e todas as seguintes



Por cada trama que chega correctamente
→ receptor envia **confirmação positiva**

A trama 3 chegou com erros
→ receptor envia **confirmação negativa**

A trama 3 ainda não chegou correctamente
→ receptor **rejeita todas as tramas seguintes**

A trama 3 chega correctamente
→ receptor envia **confirmação positiva** e aceita todas as tramas seguintes que chegarem correctamente e por ordem

Correção de erros

Go-Back-N

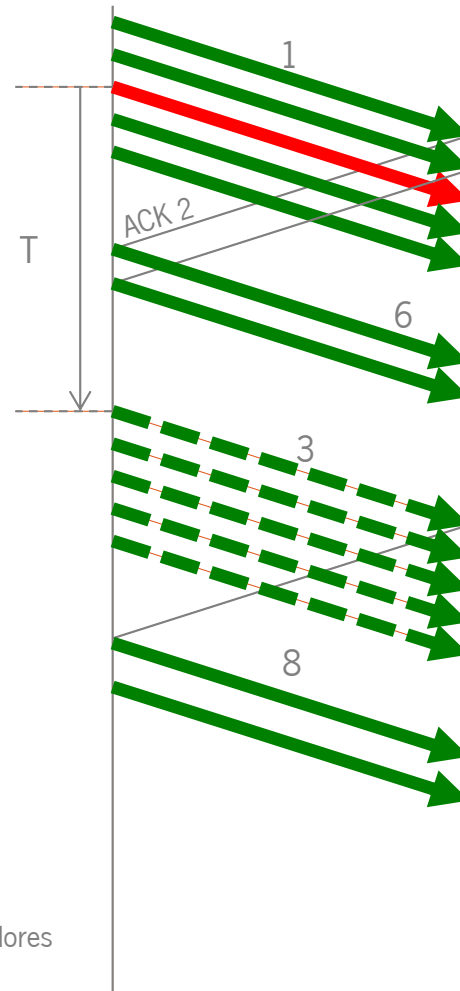


Com temporizadores⁽¹⁾

$W = 5$

Por cada confirmação positiva que chega
→ emissor envia mais uma trama, até esgotar a janela

Temporizador da trama 3 expira
→ emissor **retransmite** a trama 3, e **todas as seguintes**



Por cada trama que chega correctamente

→ receptor envia **confirmação positiva**

A trama 3 chegou com erros (ou não chegou)

→ receptor não envia nada

A trama 3 ainda não chegou correctamente

→ receptor **rejeita todas as tramas seguintes**

A trama 3 chega correctamente

→ receptor envia **confirmação positiva** e **aceita todas as tramas seguintes que chegarem correctamente e por ordem**

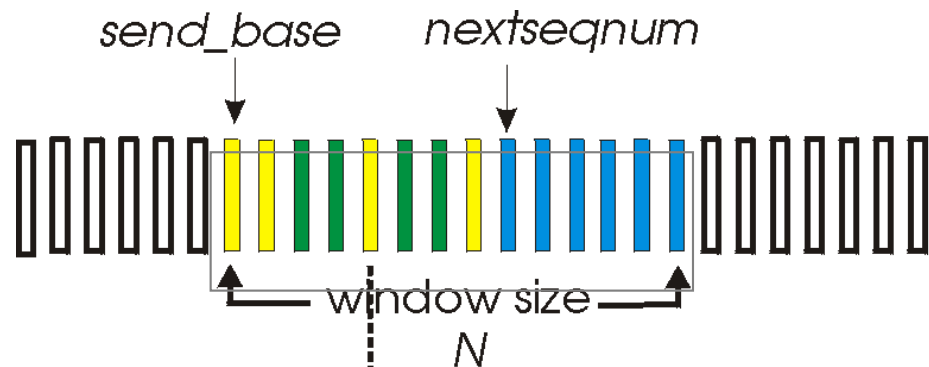
(1) Também é possível combinar temporizadores com NACKs



- ***Selective Repeat***

- apenas são retransmitidas as tramas que recebem confirmação negativa explícita ou se ocorre *timeout*.
- todas as tramas têm que ser individualmente confirmadas, embora o tamanho da janela continue a pressupor tramas consecutivas
- Implica um timer por cada trama enviada que ainda não foi confirmada
- $W_{\max}$ mais restritivo para não sobrepor as janelas na transmissão e na receção ($W_{\max}=2^{n-1}$ e não $W_{\max}=2^n-1$)
- vantagem: menos retransmissões, melhor utilização da ligação
- desvantagem: requer mais processamento (e controlo) na transmissão e na receção

Selective repeat: emitter e receptor



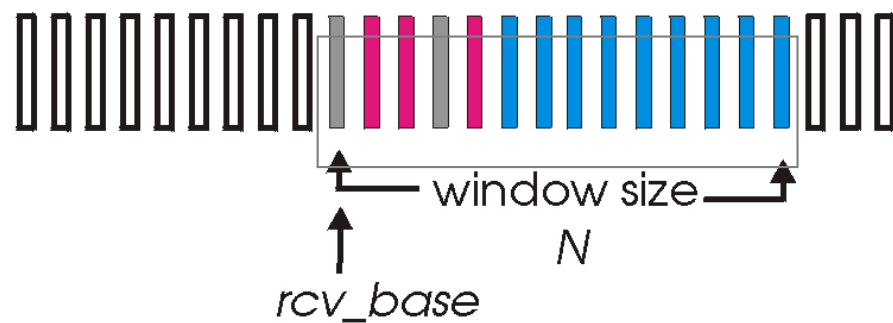
(a) sender view of sequence numbers

already
ack'ed

sent, not
yet ack'ed

usable, not
yet sent

not usable



(b) receiver view of sequence numbers

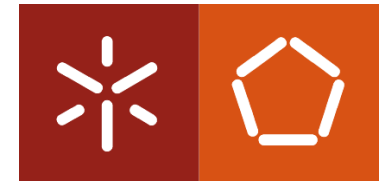
out of order
(buffered) but
already ack'ed

Expected, not
yet received

acceptable
(within window)

not usable

Selective repeat: emissor e receptor



emissor

dados para enviar:

- se há espaço na janela, envia trama

timeout(n):

- reenvia trama n; reinicializa timer

ACK(n) :

- marca n como recebida
- se n corresponde ao menor número de sequência de tramas por confirmar avança a janela para a próxima trama por confirmar

receptor

trama n:

- Envia ACK(n)
- Se na ordem esperada, entrega trama e avança janela para a próxima por confirmar
- Se fora da ordem, armazena em buffer

Correção de erros

Selective Repeat

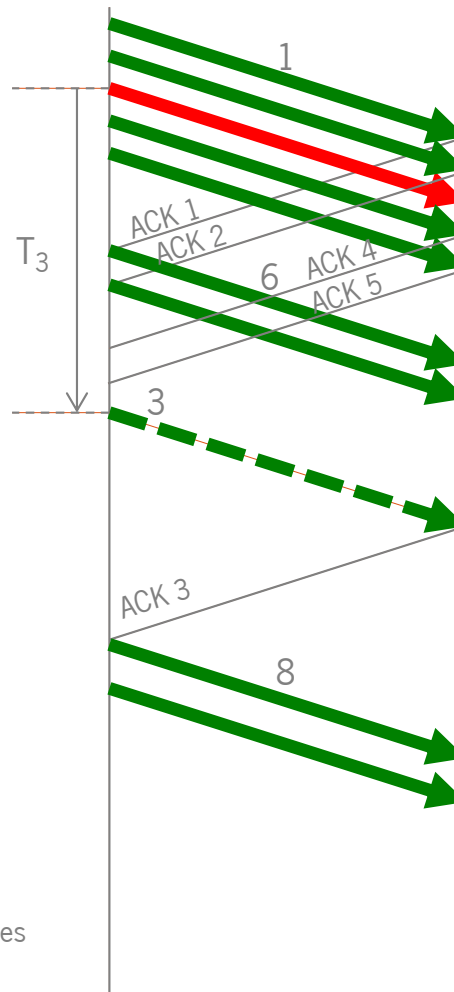


Com temporizadores⁽¹⁾

$W = 5$

Por cada confirmação positiva que chega
→ emissor envia mais uma trama, até esgotar a janela

Temporizador da trama 3 expira
→ emissor **retransmite** apenas a trama 3, e **retoma o processo no ponto em que estava antes**



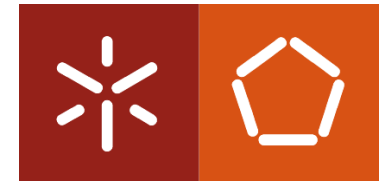
Por cada trama que chega correctamente
→ receptor envia **confirmação positiva**

A trama 3 chegou com erros (ou não chegou)
→ receptor não envia nada

A trama 3 ainda não chegou correctamente
→ receptor **aceita s tramas seguintes** que cheguem correctamente

A trama 3 chega correctamente
→ receptor envia **confirmação positiva** e **insere a trama 3 na posição correcta**

(1) Também é possível combinar temporizadores com NACKs



Para pensar:

- A estação A tem que enviar 12 tramas de dados para a estação B, através de uma ligação cujo parâmetro a é igual a 3. Suponha que a 2ª trama e 5ª trama chegam a B com erros e são transmitidas com sucesso à segunda tentativa. Através da análise dos respetivos diagramas temporais determine qual dos mecanismos de Controlo de Erros (**Go-Back-N** ou **Selective Repeat**) conduz a um menor tempo total de recepção, considerando uma janela de 5 ($W=5$).



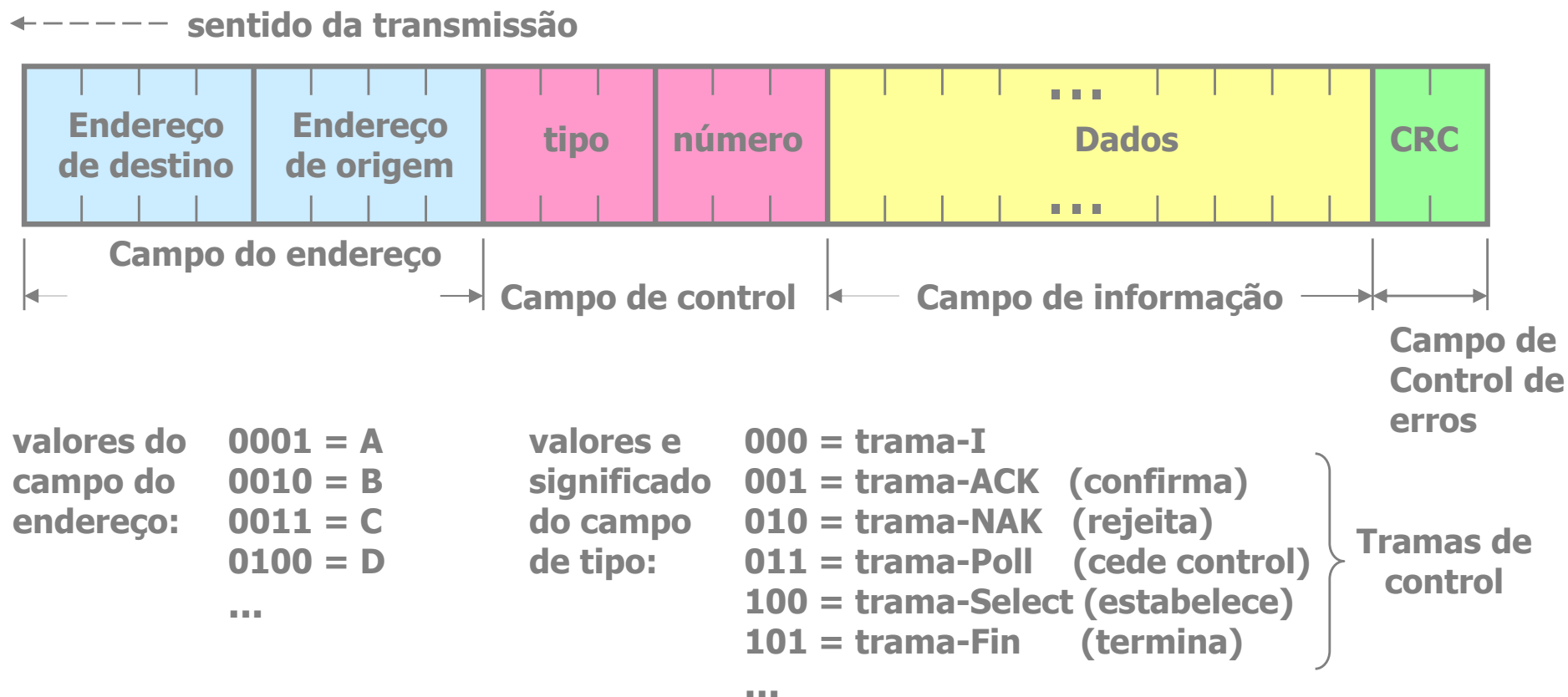
- **No mecanismo de repetição seletiva a ordem das tramas na recepção não é mantida daí que:**
 - se a ordem das tramas for relevante, o tamanho dos *buffers* pode ser inabordable
 - transmissor complexo: tem de ser capaz de enviar tramas fora de ordem
 - recetor complexo: tem de conseguir ordenar tramas
 - em geral, é usado para transmitir tramas “independentes” entre si
 - usado em meios onde a probabilidade de erro é maior (radio links)
- **O mecanismo volta-atrás-N é mais usado do que o de repetição seletiva, pois apesar de conduzir a uma pior utilização da ligação, reduz a complexidade do recetor.**

Controlo da ligação de dados

definição da trama: exemplo de um formato e semântica



- Cada protocolo define um formato de PDU, bem como os valores, o significado e o comprimento dos seus campos. Exemplo:

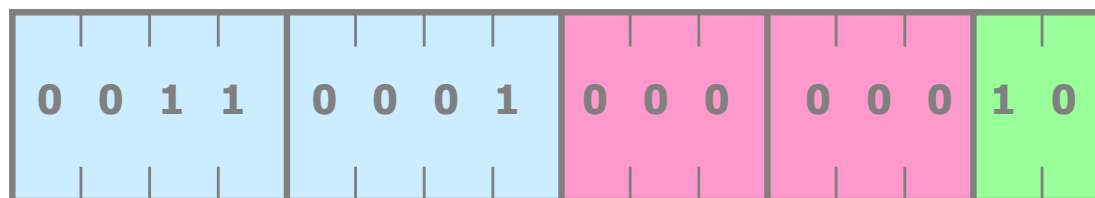


Controlo da ligação de dados

definição da trama: exemplo de um formato e semântica



- As tramas de controle não possuem o campo de dados e portanto são tramas curtas.
- Exemplo de uma trama-Select:



trama-Sel C, A, 0

Endereço
de destino

Endereço
de origem

tipo

número

CRC

(C)

(A)

(trama-Sel)

(0)

- Nesta definição protocolar está-se a pressupor que, nas tramas de resposta (ACK e NAK), o número refere-se ao de uma trama de comando do sentido oposto (recebida) que elas estão a confirmar
- Nas restantes tramas, o número representa a numeração de sequência da própria trama

Controlo da ligação de dados

protocolos (disciplinas) de linha



- **Ligações Ponto-a-Ponto (PP)**

- Em geral são ligações com um canal (circuito ou banda) para transmissão em cada sentido
- Por usarem canal dedicado (não partilhado), a ligação lógica pode efectuar-se imediatamente porque o canal está naturalmente *adquirido*.

- **Ligações Multiponto (MP)**

- Em geral são ligações com um único canal de transmissão que é partilhado por várias estações
- Duas ou mais transmissões simultâneas podem causar interferências
 - **colisão** ocorre quando uma estação recebe dois ou mais sinais ao mesmo tempo
- A ligação lógica tem de ser precedida pela aquisição do canal através de um *protocolo de acesso ao meio (protocolo MAC)*

Acesso múltiplo partilhado

- **Algoritmo distribuído que determina como é que as diferentes estações acedem ao canal, ou seja, determina quando é que uma estação pode transmitir**
- **Utiliza o próprio canal partilhado para fazer essa coordenação**



Protocolo Ideal para um canal com uma taxa de transmissão de R bps:

1. Quando apenas uma estação pretende transmitir pode fazê-lo a uma taxa **R**
2. Quando M estações pretendem transmitir cada uma dela transmite a uma taxa **R/M**
3. Completamente **distribuída**
 - Não existe uma única estação responsável por coordenar as transmissões
 - Sem necessidade de sincronizações, *clocks*, etc.
4. **Simples**

Controlo da ligação de dados

protocolo de acesso ao meio - MAC (Medium Access Control)



Taxonomia:

- **Partição do Canal**

O canal é dividido em “peças” mais pequenas (por tempo ou frequências) e cada uma das “peças” é alocada a um nó para uso exclusivo

- **Acesso Aleatório**

O canal não é dividido, permite colisões e recupera das colisões

- **Acesso sequencial**

O nós esperam pela sua vez para transmitir, mas os recursos são usados na medida das necessidades de cada um (com limites).

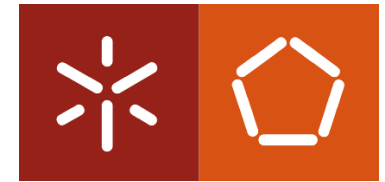
Protocolos de Acesso ao Meio

Acesso aleatório



- **Quando uma estação tem um pacote para enviar**
 - Envia-o usando toda a capacidade do canal (R)
 - Não existe coordenação entre as estações
- **Duas ou mais estações a transmitirem simultaneamente dá “colisão”**
- **Os protocolos MAC de acesso aleatório especificam:**
 - Como detectar colisões
 - Como recuperar de colisões (por exemplo através de retransmissões com atraso aleatório)
- **Exemplos de protocolos MAC de Acesso Aleatório:**
 - slotted ALOHA
 - ALOHA
 - CSMA, CSMA/CD, CSMA/CA

Slotted ALOHA



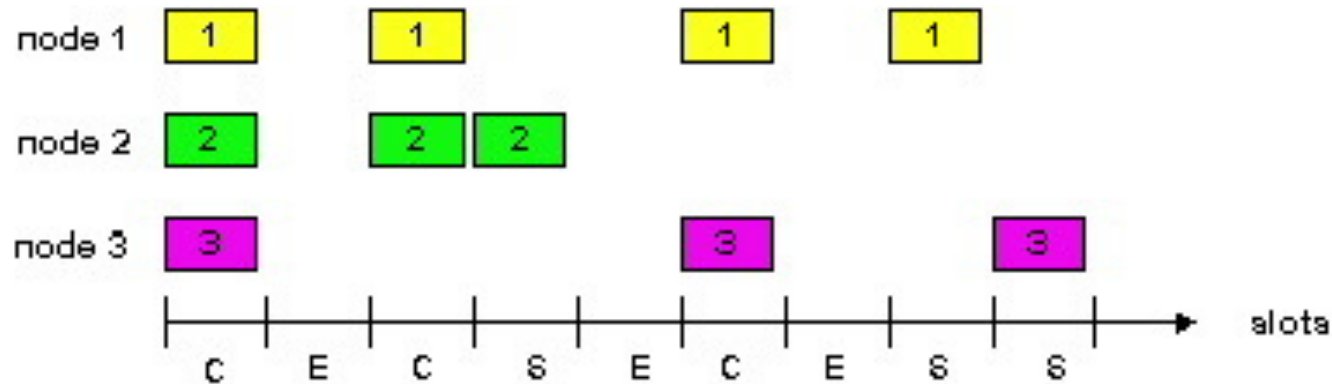
Assumindo que:

- Todas as tramas têm o mesmo tamanho
- O tempo é dividido em *slots* de igual tamanho, em cada *slot* cabe uma trama
- As estações começam a transmitir apenas no início dos *slots*
- As estações estão sincronizadas
- Se duas ou mais estações começarem a transmitir num *slot*, todas as estações detectam a colisão

Operação

- Quando uma estação tem uma trama para transmitir espera pelo início do próximo *slot* e transmiti-a
- Se não ocorrer nenhuma colisão a estação pode enviar a próxima trama no *slot* seguinte
- Se ocorrer uma colisão a estação retransmite a trama num dos próximos *slots* com uma probabilidade p

Slotted ALOHA



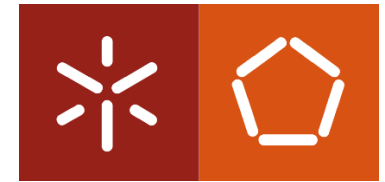
Pros

- Uma estação pode transmitir continuamente utilizando toda a capacidade do canal
- Distribuída: só os *slots* é que têm que ser sincronizados entre as estações
- Simples

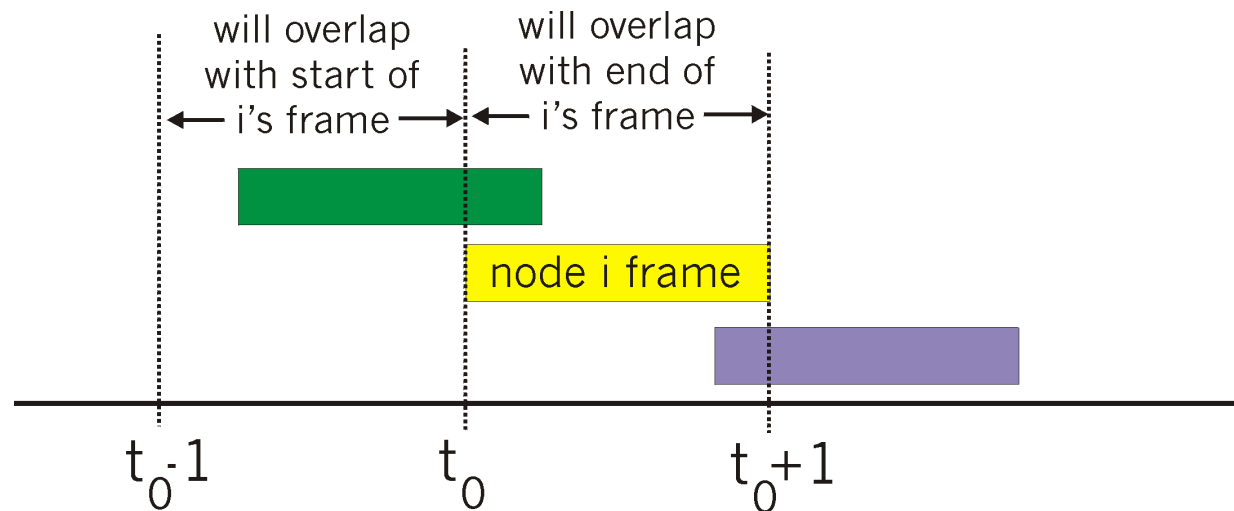
Cons

- As colisões levam a um desperdício *slots*
- *Slots* desocupados
- Os nós podem conseguir detectar uma colisão antes de acabar de transmitir a trama.
- Sincronização de *clocks*

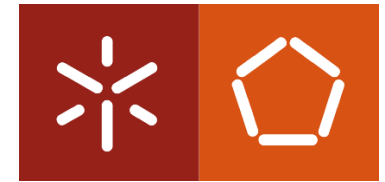
ALOHA Puro (unslotted)



- **Aloha Puro**: mais simples e sem necessidade de sincronização
- **Quando o transmissor tem um trama pronta**
 - transmiti-a imediatamente
- **A probabilidade de ocorrerem colisões aumenta**
 - Uma trama enviada em t_0 colide com outras tramas enviadas em $[t_0-1, t_0+1]$



CSMA (Carrier Sense Multiple Access)



CSMA: ouvir antes de transmitir

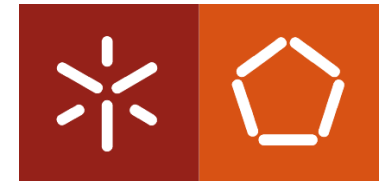
Se o canal está desocupado

- transmite a trama

Senão

- retarda a transmissão

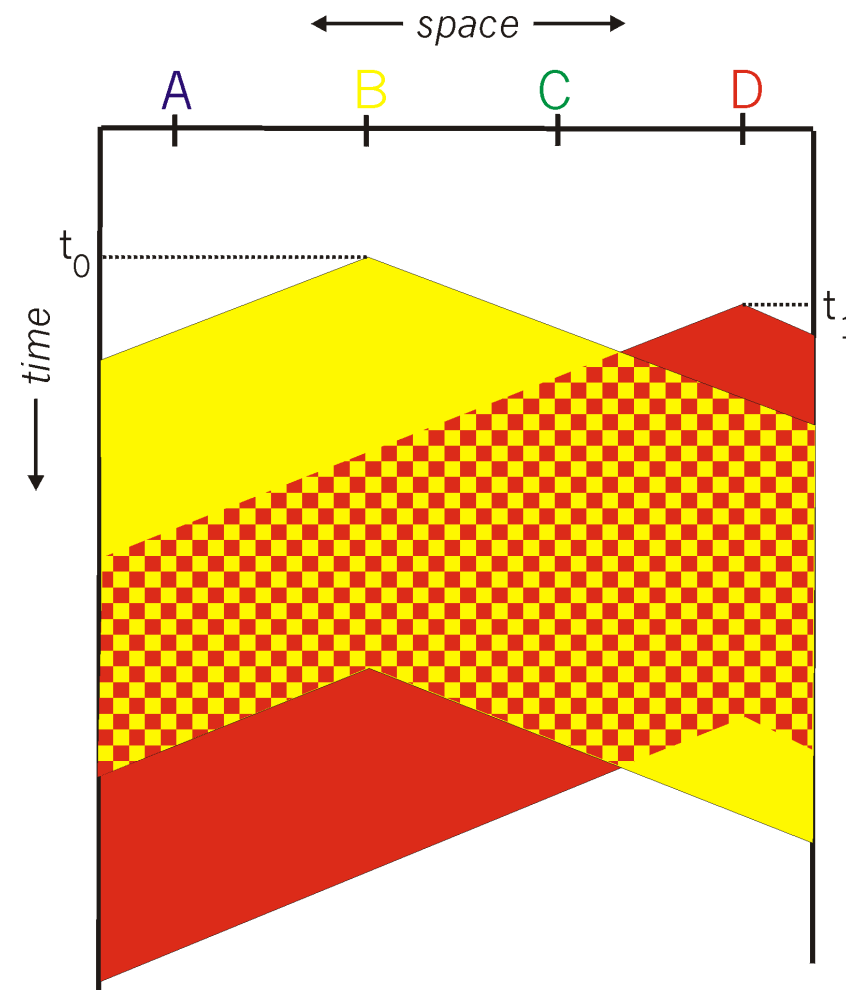
Colisões CSMA



As colisões podem acontecer na **mesma**: o atraso de propagação faz com que a estação não se aperceba que outra estação já começou a transmitir

Nota: A distância e o tempo de propagação têm um papel importante na probabilidade de ocorrerem colisões

Colisão: O tempo de transmissão do pacote é desperdiçado



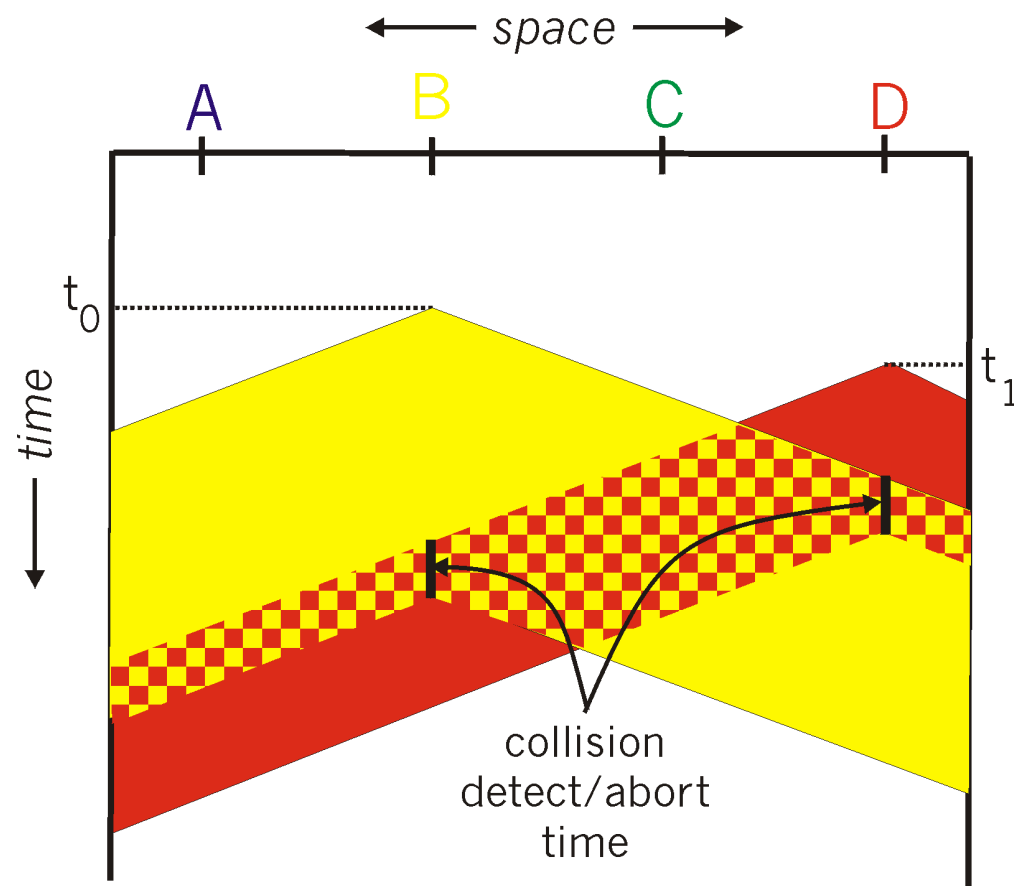
CSMA/CD (Collision Detection)

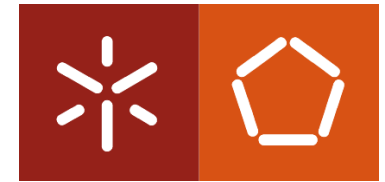


CSMA/CD: o transmissor escuta o canal antes de transmitir como no CSMA, mas depois continua à escuta enquanto transmite

- As colisões são habitualmente detectadas num curto espaço de tempo
- Quando uma colisão é detectada a transmissão é abortada para não desperdiçar a largura de banda do canal
- **A detecção das colisões:**
 - É fácil nas redes LAN com fios: o sinal recebido é comparado com o transmitido.

CSMA/CD collision detection





Protocolos MAC sequenciais

0 protocolos MAC baseados na partição dos canais

- Partilham o canal de forma justa e eficiente no caso da carga ser alta
- São ineficientes no caso da carga ser baixa: atraso no acesso ao canal, a largura de banda não excede R/N mesmo que só haja uma estação activa

Protocolos MAC de acesso aleatório

- São eficientes no caso da carga ser baixa, uma estação pode disfrutar de toda a capacidade do canal
- No caso da carga ser alta o excesso de colisões traz uma grande sobrecarga

Protocolos MAC de acesso sequencial

- Tentam “apanhar o melhor dos dois mundos”

Protocolos MAC sequenciais



Polling:

- **Estação primária convida as estações secundárias para transmitir à vez**
- **Contras:**
 - Sobrecarga do *polling*
 - Latência
 - Um único ponto de falha (estação primária)

Passagem de testemunho (*token*):

- **O testemunho é passado de uma estação para a estação seguinte de forma sequencial.**
- **Contras:**
 - Sobrecarga do *token*
 - Latência
 - Um único ponto de falha (*token*)

